Chapter 19: Stacks and Queues

19.1 Introduction to the Stack ADT

Concept:

A stack is a data structure that stores and retrieves items in a last-in, first-out manner.

Definition

A stack is a data structure, similar to an array or linked list, that holds a sequence of elements. Stacks operate on a **last-in, first-out (LIFO)** basis, meaning the last element added to the stack is the first one to be removed. An everyday analogy is a stack of cafeteria plates: the first plate placed on the stack is the last one to be used.

Applications of Stacks

Stacks are ideal for algorithms that process the most recently saved element first. Computer systems use stacks during program execution to manage function calls. When a function is called, the system saves the program's return address on a stack. Local variables for functions are also created on a stack. When the function ends, its local variables are removed, and the return address is retrieved so execution can continue. Some calculators also utilize a stack for mathematical computations.

Static and Dynamic Stacks

- **Static stacks**: Have a fixed size and are typically implemented as arrays.
- **Dynamic stacks**: Can grow or shrink in size as needed and are usually implemented as linked lists.

Stack Operations

A stack's two main operations are **push** and **pop**.

The **push** operation adds a value onto the top of the stack.

```
push(5);
push(10);
push(15);
```

The **pop** operation retrieves and removes a value from the top of the stack.

A static stack requires a Boolean `isFull` operation to prevent stack overflow by checking if the stack is at capacity. Both static and dynamic stacks need a Boolean `isEmpty` operation to prevent errors when attempting to pop from an empty stack.

A Static Stack Implementation

We will examine a static stack for integers implemented using a `struct` and a set of free functions.

Table 19-1 The `IntStack` Struct's Members

MEMBER	DESCRIPTION
`stackArray`	A pointer to `int`. When the stack is initialized, it uses `stackArray` to dynamically allocate an array for storage.
`stackSize`	An integer that holds the size of the stack.
`top`	An integer that is used to mark the top of the stack.

Table 19-2 The `IntStack` Functions

FUNCTION	DESCRIPTION
`initStack`	Accepts an integer argument for the stack's size, allocates memory for it, and initializes `top` to `-1`.
`destroyStack`	Frees the memory that was allocated by `initStack`.
`isFull`	Returns `true` if the stack is full (`top == stackSize - 1`), and `false` otherwise.
`isEmpty`	Returns `true` if the stack is empty (`top == -1`), and `false` otherwise.

FUNCTION	DESCRIPTION
`pop`	Removes the value at the top of the stack and returns it through a reference parameter.
`push`	Accepts an integer argument and pushes it onto the top of the stack.

Note:

Even though the `initStack` function dynamically allocates the stack array, this is still a static stack because its size does not change after allocation.

A Procedural Implementation of a Static Integer Stack

```
#include <iostream>

struct IntStack
{
    int *stackArray;
    int stackSize;
    int top;
};

void initStack(IntStack* stack, int size);
void destroyStack(IntStack* stack);
void push(IntStack* stack, int num);
void pop(IntStack* stack, int &num);
bool isFull(const IntStack* stack);
bool isEmpty(const IntStack* stack);

void initStack(IntStack* stack, int size)
{
    stack->stackArray = new int[size];
    stack->stackSize = size;
    stack->top = -1;
}

void destroyStack(IntStack* stack)
{
    if (stack->stackSize > 0)
        delete[] stack->stackArray;
}
```

```
void push(IntStack* stack, int num)
{
    if (isFull(stack))
    {
        std::cout << "The stack is full.\n";
    }
    else
    {
        stack->top++;
        stack->stackArray[stack->top] = num;
    }
}

void pop(IntStack* stack, int &num)
{
    if (isEmpty(stack))
    {
        std::cout << "The stack is empty.\n";
    }
    else
    {
        num = stack->stackArray[stack->top];
        stack->top--;
    }
}

bool isFull(const IntStack* stack)
{
    return stack->top == stack->stackSize - 1;
}

bool isEmpty(const IntStack* stack)
{
    return stack->top == -1;
}
```

The `top` member holds the subscript of the last element, effectively marking the top of the stack. When `top` is -1, the stack is considered empty (`isEmpty` returns `true`). The stack is full when `top` equals `stackSize - 1` (`isFull` returns `true`).

Program 19-1

This program demonstrates the procedural `IntStack` implementation.

```
#include <iostream>


int main()
{
    int catchVar;
    IntStack stack;

    initStack(&stack, 5);

    std::cout << "Pushing 5\n";
    push(&stack, 5);
    std::cout << "Pushing 10\n";
    push(&stack, 10);
    std::cout << "Pushing 15\n";
    push(&stack, 15);
    std::cout << "Pushing 20\n";
    push(&stack, 20);
    std::cout << "Pushing 25\n";
    push(&stack, 25);

    std::cout << "Popping...\n";
    pop(&stack, catchVar);
    std::cout << catchVar << std::endl;
    pop(&stack, catchVar);
    std::cout << catchVar << std::endl;
    pop(&stack, catchVar);
    std::cout << catchVar << std::endl;
    pop(&stack, catchVar);
    std::cout << catchVar << std::endl;
    pop(&stack, catchVar);
    std::cout << catchVar << std::endl;

    destroyStack(&stack);

    return 0;
}
```

💻 Program Output

Initially, with `top` as -1, the stack is empty. After the first `push` operation, `top` moves to element 0. After five `push` calls, `top` is at element 4, and the stack is full. The `pop` function uses a reference parameter `num` to return the popped value.

Implementing Other Stack Operations

More complex operations can be built using the basic stack functions. For example, we can create `add` and `sub` functions that operate on an `IntStack`.

- `add()`: Pops two values, adds them, and pushes the sum back onto the stack.
- `sub()`: Pops two values, subtracts the second from the first, and pushes the difference.

Math Functions for an Integer Stack

```
void add(IntStack* stack)
{
    int num, sum;

    pop(stack, sum);
    pop(stack, num);

    sum += num;

    push(stack, sum);
}

void sub(IntStack* stack)
{
    int num, diff;

    pop(stack, diff);
    pop(stack, num);

    diff -= num;
```

```
    push(stack, diff);
}
```

Program 19-2

This program demonstrates the math functions with the `IntStack`.

```
#include <iostream>


int main()
{
    int catchVar;
    IntStack stack;

    initStack(&stack, 5);

    std::cout << "Pushing 3\n";
    push(&stack, 3);
    std::cout << "Pushing 6\n";
    push(&stack, 6);

    add(&stack);

    std::cout << "The sum is ";
    pop(&stack, catchVar);
    std::cout << catchVar << std::endl << std::endl;

    std::cout << "Pushing 7\n";
    push(&stack, 7);
    std::cout << "Pushing 10\n";
    push(&stack, 10);

    sub(&stack);

    std::cout << "The difference is ";
    pop(&stack, catchVar);
    std::cout << catchVar << std::endl;

    destroyStack(&stack);

    return 0;
}
```

💻 **Program Output**

```
```

19.2 Dynamic Stacks

Concept:

A stack may be implemented as a linked list to expand or shrink with each `push` or `pop` operation.

A dynamic stack is built on a linked list instead of an array. Its advantages over an array-based stack include not needing a predefined size and never becoming full (as long as the system has available memory). Here, we will look at a dynamic stack implementation for integers.

A Procedural Implementation of a Dynamic Integer Stack

```
#ifndef DYNINTSTACK_H
#define DYNINTSTACK_H
#include <iostream>

struct StackNode
{
    int value;
    StackNode *next;
};

struct DynIntStack
{
    StackNode *top;
};

void initStack(DynIntStack*);
void destroyStack(DynIntStack*);
```

```
void push(DynIntStack*, int);
void pop(DynIntStack*, int &);
bool isEmpty(const DynIntStack*);

void initStack(DynIntStack* stack)
{
    stack->top = nullptr;
}

void destroyStack(DynIntStack* stack)
{
    StackNode *nodePtr = stack->top, *nextNode = nullptr;
    while (nodePtr != nullptr)
    {
        nextNode = nodePtr->next;
        delete nodePtr;
        nodePtr = nextNode;
    }
    stack->top = nullptr;
}

void push(DynIntStack* stack, int num)
{
    StackNode *newNode = new StackNode;
    newNode->value = num;
    newNode->next = stack->top;
    stack->top = newNode;
}

void pop(DynIntStack* stack, int &num)
{
    if (isEmpty(stack))
    {
        std::cout << "The stack is empty.\n";
    }
    else
    {
        num = stack->top->value;
        StackNode *temp = stack->top;
        stack->top = stack->top->next;
        delete temp;
    }
}
```

```
bool isEmpty(const DynIntStack* stack)
{
    return stack->top == nullptr;
}
#endif
```

The `StackNode` struct defines each node in the linked list. A `top` pointer, analogous to a linked list's `head` pointer, always points to the first node, which represents the stack's top. An initial `nullptr` value indicates an empty stack.

- `push` **operation**: A new node is allocated and its `value` member is set. The new node is inserted at the head of the list, and `top` is updated to point to it.
- `pop` **operation**: If the stack is not empty, `pop` removes the node at the head of the list. The value from that node is copied into the reference parameter `num`, and the `top` pointer is updated to point to the next node.
- `isEmpty` **operation**: Returns `true` if the `top` pointer is `nullptr`.

Program 19-4

This program demonstrates the dynamic stack implementation.

```
#include <iostream>
#include "DynIntStack.h"

int main()
{
    int catchVar;
    DynIntStack stack;

    initStack(&stack);

    std::cout << "Pushing 5\n";
    push(&stack, 5);
    std::cout << "Pushing 10\n";
    push(&stack, 10);
    std::cout << "Pushing 15\n";
    push(&stack, 15);

    std::cout << "Popping...\n";
    pop(&stack, catchVar);
    std::cout << catchVar << std::endl;
    pop(&stack, catchVar);
```

```
    std::cout << catchVar << std::endl;
    pop(&stack, catchVar);
    std::cout << catchVar << std::endl;

    std::cout << "\nAttempting to pop again... ";
    pop(&stack, catchVar);

    destroyStack(&stack);
    return 0;
}
```

Program Output

```

```

19.3 Introduction to the Queue ADT

Concept:

A queue is a data structure that stores and retrieves items in a `first-in, first-out` manner.

Definition

A queue is a data structure that, like a stack, holds a sequence of elements. However, it provides access in a **first-in, first-out (FIFO)** order. This behavior is similar to a checkout line at a grocery store, where the first customer to arrive is the first one served.

Application of Queues

Queues are frequently used in computer operating systems, especially in multi-user or multi-tasking environments, to manage access to shared resources. For instance, a print queue holds documents, allowing the printer to service them one at a time. Communications

software also uses queues to buffer data received over networks, which is particularly useful when data arrives faster than it can be processed.

Static and Dynamic Queues

Like stacks, queues can be implemented using either arrays (static queues) or linked lists (dynamic queues). Dynamic queues offer the same advantages over static queues as dynamic stacks do over static stacks, with the primary difference being the element access pattern.

Queue Operations

Queues can be conceptualized as having a **front** and a **rear**.

The two main queue operations are **enqueuing** and **dequeuing**.

- **Enqueue**: To insert an element at the rear of a queue.
- **Dequeue**: To remove an element from the front of a queue.

There are various algorithms to implement these operations. The following example shows several enqueue operations on an empty static queue.

```
enqueue(3);
enqueue(6);
enqueue(9);
```

In a simple implementation, the `front` index remains fixed while the `rear` index moves forward as items are added. Dequeuing would involve removing the front element and shifting all subsequent elements forward by one position. This shifting algorithm is inefficient because the time it takes to dequeue increases with the number of items in the queue.

A more efficient method allows both the `front` and `rear` indices to move. When an item is dequeued, the `front` index simply moves one position toward the rear, eliminating the need for a costly shifting operation. However, this approach causes the queue's contents to gradually "crawl" toward the end of the array.

The problem with the crawling approach is that the `rear` index cannot move past the end of the array. The solution is to treat the array as **circular**. When an index moves past the end, it "wraps around" to the beginning.

For example, if the next enqueue operation inserts the value 4 into a full queue, the rear of the queue wraps around to the start of the array.

This wrap-around logic can be implemented with an `if` statement or more elegantly with modular arithmetic.

```
if (rear == queueSize - 1)
   rear = 0;
else
   rear++;
```

```
rear = (rear + 1) % queueSize;
```

Detecting Full and Empty Queues with Circular Arrays

A challenge with the circular array method is distinguishing between a full and an empty queue, as the front and rear indices might point to the same location in some scenarios. Two common solutions are to always leave one empty element between the rear and front indices or to use a counter variable to track the total number of items in the queue. We will use the counter variable method in our implementation.

A Static Queue Implementation

The `IntQueue` implementation below uses a `struct` and a set of functions. It also includes a `clear` function to reset the queue.

A Procedural Implementation of a Static Integer Queue

```
#ifndef INTQUEUE_H
#define INTQUEUE_H
#include <iostream>

struct IntQueue
{
    int *queueArray;
    int queueSize;
    int front;
    int rear;
    int numItems;
};

void initQueue(IntQueue*, int);
void destroyQueue(IntQueue*);
void enqueue(IntQueue*, int);
void dequeue(IntQueue*, int &);
```

```
bool isEmpty(const IntQueue*);
bool isFull(const IntQueue*);
void clear(IntQueue*);

void initQueue(IntQueue* q, int s)
{
    q->queueArray = new int[s];
    q->queueSize = s;
    q->front = -1;
    q->rear = -1;
    q->numItems = 0;
}

void destroyQueue(IntQueue* q)
{
    delete [] q->queueArray;
}

void enqueue(IntQueue* q, int num)
{
    if (isFull(q))
    {
        std::cout << "The queue is full.\n";
    }
    else
    {
        q->rear = (q->rear + 1) % q->queueSize;
        q->queueArray[q->rear] = num;
        q->numItems++;
    }
}

void dequeue(IntQueue* q, int &num)
{
    if (isEmpty(q))
    {
       std::cout << "The queue is empty.\n";
    }
    else
    {
        q->front = (q->front + 1) % q->queueSize;
        num = q->queueArray[q->front];
        q->numItems--;
    }
```

```
}

bool isEmpty(const IntQueue* q)
{
    return q->numItems == 0;
}

bool isFull(const IntQueue* q)
{
    return q->numItems >= q->queueSize;
}

void clear(IntQueue* q)
{
    q->front = q->queueSize - 1;
    q->rear = q->queueSize - 1;
    q->numItems = 0;
}
#endif
```

Program 19-7

```
#include <iostream>
#include "IntQueue.h"

int main()
{
    const int MAX_VALUES = 5;
    IntQueue iQueue;
    initQueue(&iQueue, MAX_VALUES);

    std::cout << "Enqueuing " << MAX_VALUES << " items...\n";
    for (int x = 0; x < MAX_VALUES; x++)
        enqueue(&iQueue, x);

    std::cout << "Now attempting to enqueue again...\n";
    enqueue(&iQueue, MAX_VALUES);

    std::cout << "The values in the queue were:\n";
    while (!isEmpty(&iQueue))
    {
        int value;
        dequeue(&iQueue, value);
        std::cout << value << std::endl;
```

```
    }

    destroyQueue(&iQueue);
    return 0;
}
```

💻 **Program Output**

```

```

19.4 Dynamic Queues

Concept:

A queue may be implemented as a linked list to expand or shrink with each enqueue or dequeue operation.

Dynamic queues, implemented with linked lists, are often more intuitive than static queues. A dynamic queue begins as an empty linked list.

- **Enqueue**: A new node is added to the rear of the list. The `rear` pointer is updated to this new node. If it's the first node, the `front` pointer also points to it.
- **Dequeue**: The node at the front of the list is deleted. The `front` pointer is updated to point to the next node.

Below is the code for a dynamic integer queue implemented procedurally.

A Procedural Implementation of a Dynamic Integer Queue

```
#ifndef DYNINTQUEUE_H
#define DYNINTQUEUE_H
#include <iostream>
```

```
struct QueueNode
{
    int value;
    QueueNode *next;
};

struct DynIntQueue
{
    QueueNode *front;
    QueueNode *rear;
    int numItems;
};

void initQueue(DynIntQueue*);
void clear(DynIntQueue*);
void enqueue(DynIntQueue*, int);
void dequeue(DynIntQueue*, int &);
bool isEmpty(const DynIntQueue*);

void initQueue(DynIntQueue* q)
{
    q->front = nullptr;
    q->rear = nullptr;
    q->numItems = 0;
}

void clear(DynIntQueue* q)
{
    int value;
    while(!isEmpty(q))
        dequeue(q, value);
}

void enqueue(DynIntQueue* q, int num)
{
    QueueNode *newNode = new QueueNode;
    newNode->value = num;
    newNode->next = nullptr;

    if (isEmpty(q))
    {
        q->front = newNode;
        q->rear = newNode;
    }
```

```
    else
    {
        q->rear->next = newNode;
        q->rear = newNode;
    }
    q->numItems++;
}

void dequeue(DynIntQueue* q, int &num)
{
    if (isEmpty(q))
    {
        std::cout << "The queue is empty.\n";
    }
    else
    {
        num = q->front->value;
        QueueNode *temp = q->front;
        q->front = q->front->next;
        delete temp;
        q->numItems--;
    }
}

bool isEmpty(const DynIntQueue* q)
{
    return q->numItems == 0;
}
#endif
```

Program 19-9

```
#include <iostream>
#include "DynIntQueue.h"

int main()
{
    const int MAX_VALUES = 5;
    DynIntQueue iQueue;
    initQueue(&iQueue);

    std::cout << "Enqueuing " << MAX_VALUES << " items...\n";
    for (int x = 0; x < MAX_VALUES; x++)
        enqueue(&iQueue, x);
```

```
    std::cout << "The values in the queue were:\n";
    while (!isEmpty(&iQueue))
    {
        int value;
        dequeue(&iQueue, value);
        std::cout << value << std::endl;
    }

    clear(&iQueue);

    return 0;
}
```

💻 Program Output